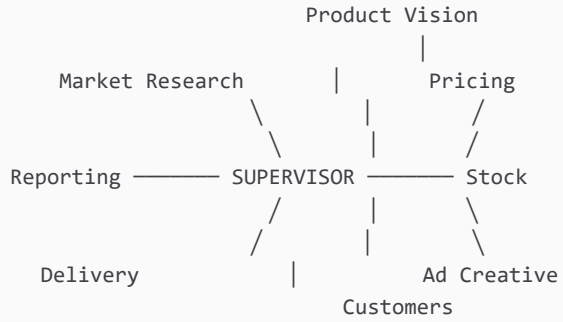
Lucida

An AI workforce for a small shop — how the parts fit together, what each specialist does, and which external service is called for what.

Technical reference · One supervisor, eight specialists, 22 database tables, 47 routes

1 · The shape of the thing

Lucida is a **star**, not a pipeline. Every request enters at the supervisor, which reads it, plans, and hands the work to exactly one specialist. That specialist does its job, writes what it learned into shared memory, and returns to the supervisor, which either picks the next specialist or finishes and writes the answer.



The star matters. A pipeline would run all eight agents for every question, which is slow and expensive; a question about stock levels has no business waking the ad writer. The supervisor decides, and the route it actually took is recorded and replayable.

What travels between them

A single state object moves through the graph, carrying the owner's request, any image paths, the plan, which agent is next and why, and an accumulating record of every agent's output and errors. Agents never talk to each other directly — they read the state and write to it, which is what makes any run replayable step by step afterwards.

Where it stops

Anything irreversible or costly — publishing an ad, spending money, booking a courier — suspends the graph at an interrupt and waits. The owner sees it on the dashboard. Approving *resumes from that exact point* rather than starting the run again, because the graph's state is checkpointed to SQLite.

> **Why the owner is a node in the system, not a spectator.** The gates are not a confirmation dialog bolted on at the end. They are edges in the graph: the run genuinely cannot proceed past one without a decision, and the decision is recorded alongside the work it authorised.

2 · The eight specialists

Market Research reads the web

Finds what sells, what rivals charge, and what people are asking for that nobody supplies.

Calls	DuckDuckGo (or Tavily if a key is set) — several searches per run: competitors, demand signals, price bands, local posts
Writes	`competitors` — name, price, source URL, but only for results that actually name a price
Note	Searches the shop's own country index; an untargeted search for "price in Dhaka" returns Amazon and Etsy

Product Vision reads photos

Turns a photograph of a product into a go/no-go call: what it is, what it is made of, what it could sell for, and whether it is worth making.

Calls	Google Gemini (multimodal) for the image; web search to sanity-check the category and price band
Writes	`products`, `media_assets`
Note	Runs on Google specifically because Groq serves no vision model. Without a Google key it says it could not look at the photo rather than guessing from the filename

Pricing & Cost does the arithmetic

Unit cost, sell price, margin percentage, break-even volume, and the reasoning behind the number.

Calls	A restricted Python sandbox — the sums are executed, not generated by a language model
Writes	`pricing_history`; updates `products`
Note	May price a product the shop does not yet sell, but cannot create it. A recommendation is advice; the catalogue is a record of what exists

Stock counts things

What is on hand, what it cost, what runs out first, and what to reorder.

Calls	No external service — reads and writes the shop's own database
Writes	`products`, `inventory`, `stock_movements`
Note	Records a weight when the owner mentions one, because weight is what every delivery quote is computed from

Ad Creative writes and publishes

Platform-specific ad copy — different for Facebook, Instagram and YouTube — and the artwork to go with it.

Calls	Pollinations or Gemini for images; Meta Graph API and YouTube Data API to publish
Writes	`campaigns`, `social_posts`, `media_assets`
Gate	Publishing always waits for approval. It spends the owner's money and their reputation

Customers the listening loop

Reads DMs and comments, works out sentiment and intent, extracts pre-orders, spots demand for things the shop does not sell, and drafts replies for the owner to send.

Calls	Meta Graph API (Messenger and Instagram) to read and reply
Writes	`social_messages`, `conversations`, `preorders`
Note	Each draft is matched back to its message by id. A draft that cannot be placed is discarded, never sent to a customer it might not answer

Delivery books parcels

Quotes a parcel by weight and destination, then books the courier and arranges cash-on-delivery collection.

Calls	Pathao and Steadfast — token, price plan, order creation, tracking
Writes	`deliveries`, `addresses`
Gate	Booking waits for approval: it dispatches a real rider and commits the shop to a customer

Reporting closes the loop

Reads across every other agent's work and writes the day up in the owner's own numbers — what sold, what it cost, what to do next.

Calls	The sandbox for figures; the shop's whole database
Writes	`reports`; semantic memory, so later runs recall earlier conclusions
Note	The heaviest reader of shared memory — this is where the cycle becomes a loop rather than a line

3 · Which service, and what for

Every external call has one job. Nothing is called speculatively, and anything unavailable degrades to a labelled stand-in rather than a failure.

Language and vision

SERVICE	USED FOR	COST	STATE
Groq `api.groq.com`	Every text decision: the supervisor's routing and planning, and seven of the eight specialists' reasoning	Free tier, daily cap per model	live
Google Gemini `generativelanguage.googleapis.com`	Reading product photographs, and the fallback for text when Groq's daily allowance is spent	Free tier	live

> **Failover is not optional here.** Free tiers are rationed and providers retire models without warning — both happened during this project. The chain moves on when a model is *capped* (429), *retired* (404), or *returns the right JSON but not as a tool call*, which Groq rejects server-side and which used to kill an entire run. Only when everything is exhausted does the owner see a message, and it says when to come back.

Search and reference

SERVICE	USED FOR	COST	STATE
DuckDuckGo	Competitor and demand research, targeted at the shop's own country index	Free, no account	live
Tavily	The same, but returns extracted page content instead of a one-line snippet — which is what lets prices be picked out of results	Free tier	key needed
open.er-api.com	Currency conversion, so costs quoted in dollars can be shown in taka	Free, no account	live

Images

SERVICE	USED FOR	COST	STATE
Pollinations	Ad artwork. Gets the colour and setting right; often invents the object itself	Free, no account	live
Gemini image models	Better ad artwork — tried first when a key is present	Needs billing; free keys get zero image quota	billing needed

Selling channels

SERVICE	USED FOR	GATE	STATE
Meta Graph API Messenger, Facebook, Instagram	Reading customer DMs and comments, sending replies, publishing posts and ads	App Review for messaging permissions	pending
YouTube Data API v3	Publishing videos the owner supplies	OAuth refresh token — an API key cannot upload	pending

Couriers

SERVICE	USED FOR	NOTES	STATE
Pathao `api-hermes.pathao.com`	Issuing a token, listing stores, resolving city and zone, pricing a parcel from Pathao's own rate card, creating the consignment	Token needs the client pair *and* the merchant login together	live
Steadfast `portal.packzy.com`	Creating a consignment, checking balance, tracking	API key and secret from the merchant portal	key needed

Infrastructure

SERVICE	USED FOR
SMTP (Gmail or any)	Sending six-digit verification codes. Without it the code prints on screen — fine for one owner, not for strangers signing up
Google OAuth	Sign in with Google, and the consent flow that produces a YouTube refresh token

4 · Memory, and why isolation is by file

Each shop gets its own directory: one SQLite database of 22 tables, and one file of semantic memory. Every agent shares a single `memory` object; the web layer rebinds it to the signed-in owner's directory before any agent runs.

PATH	HOLDS
`data/accounts.db`	Owners: email, bcrypt hash, business, verification state
`data/shops/<id>/shop.db`	One shop's entire business, alone
`data/shops/<id>/knowledge.jsonl`	What agents concluded, for later recall
`data/lucida.db`	The trace — every step every agent has taken
`data/checkpoints.db`	Graph state, so a run paused at a gate can resume

> **Why files rather than an owner column.** Filtering by `owner_id` works until one query forgets to. Isolation by file means no query has to remember: a missed filter cannot leak one shop into another, because the other shop is not in the database being read. The trace is the exception — one file for the whole machine — so session ids carry a hash of the owner and every dashboard query matches on that prefix.

5 · Watching it work

The Workforce screen shows a run as it happens. One connection stays open (`/api/events`) and each step is pushed the moment it is recorded — a card lights when its agent starts, handoffs appear as numbered arrows in the order they occurred, and every step scrolls into a feed with its timestamp and cost.

The same screen lets the owner hand a job to one named specialist. A directed request outranks the supervisor for exactly one hop, then the run continues normally — so it cannot loop back to the same agent forever.

6 · What is honest about it

AREA	STATE
Chat, history, photo reading, research, products, delivery quoting and booking, the live graph, admin	Working, verified end to end against live APIs
Meta, YouTube, Steadfast, email	Built and tested; waiting on credentials, not code
Ad artwork	Weak while it runs on the free keyless model. Uploading your own photograph is the reliable path
Autopilot permission switches	Present on the settings screen but not yet enforced anywhere

Anything not connected runs against a **simulated** adapter that is labelled as such everywhere it appears, and nothing simulated is written to the shop's records as though a real customer sent it.

Lucida · technical reference · generated from the codebase